المدرسة العليا للتكنولوجيا
Ecole Supérieure de Technologie
Sidi Bennour - سيدي بنور

جامعة شعيب الدكالي
Université Chouaïb Doukkali

PlantDoc AI System

Diagnostic Automatisé des Pathologies Végétales

Solution Web Intégrée pour l'Identification en Temps Réel

Étude d'Impact : Comparaison des Performances SVM (LBP) vs CNN

Rapport de Projet de Module

Module : **Gestion de Projet**

2ème année DUT | Génie Informatique

Réalisé par :

Abderrahmane Aboutalib

Anas ElMarihi • **Mohammed Bziz**

Sous la direction de :

Pr. **AL SEMAA**

Année Universitaire : 2025 – 2026

Ce rapport présente le développement du système **PlantDoc AI**, une solution complète de diagnostic automatisé des maladies végétales basée sur l'apprentissage automatique. Le projet implémente et compare deux approches : (1) un classifieur **SVM** exploitant des descripteurs **LBP**, et (2) un réseau **CNN MobileNetV2** par apprentissage par transfert. La solution intègre un pipeline complet : du jeu de données *PlantVillage* jusqu'à une interface web d'inférence en temps réel via Flask et ngrok.

Mots-Clés : Apprentissage automatique · Vision par ordinateur · CNN · SVM · LBP · MobileNetV2 · Transfer Learning · PlantVillage · Flask API · Interface Web

0 1

Contents

1 Introduction et Contexte du Projet **4**

1.1 Problématique 4

1.2 Objectifs du Projet 4

1.3 Architecture Globale du Système 4

2 Jeu de Données et Environnement **5**

2.1 Le Dataset PlantVillage 5

2.1.1 Présentation 5

2.1.2 Justification du Choix 5

2.2 Environnement d'Exécution 6

2.2.1 Mise en Place de l'Environnement 6

3 Prétraitement et Préparation des Données **7**

3.1 Pipeline de Prétraitement 7

3.1.1 Filtrage et Organisation 7

3.1.2 Générateurs de Données 7

3.1.3 Justification des Choix 7

4 Modèle Classique : SVM avec Descripteurs LBP **8**

4.1 Fondements Théoriques 8

4.1.1 Local Binary Patterns (LBP) 8

4.1.2 Support Vector Machine (SVM) 8

4.2 Implémentation 9

4.3 Analyse des Limitations 9

5 Modèle Deep Learning : CNN MobileNetV2 **10**

5.1 Architecture MobileNetV2 10

5.1.1 Principe du Transfer Learning 10

5.1.2 Architecture Spécifique au Projet 10

5.1.3 Détail des Couches 11

5.2 Implémentation et Entraînement 11

5.3 Analyse des Courbes d'Entraînement 12

6 Évaluation et Comparaison des Modèles **13**
6.1 Visualisation des Performances . . . 13
6.2 Résultats et Analyse . . . 13
6.3 Discussion des Résultats . . . 13

7 Déploiement Web – Interface PlantVision **15**
7.1 Architecture du Système de Déploiement . . . 15
7.2 Backend Flask – Serveur d'Inférence . . . 15
7.3 Frontend Web – Interface PlantVision . . . 16
7.3.1 Fonctionnalités Implémentées . . . 16
7.3.2 Communication Frontend ↔ Backend . . . 17
7.3.3 Design et Expérience Utilisateur . . . 17

8 Conclusion et Perspectives **18**
8.1 Bilan du Projet . . . 18
8.2 Leçons Apprises . . . 18
8.3 Perspectives d'Amélioration . . . 18

1 1

Introduction et Contexte du Projet

1.1 Problématique

Les maladies des plantes représentent une menace majeure pour la sécurité alimentaire mondiale. Selon l'Organisation des Nations Unies pour l'Alimentation et l'Agriculture (FAO), les ravageurs et maladies des cultures causent des pertes annuelles estimées à **40%** de la production agricole mondiale. Le diagnostic précoce et précis de ces pathologies est donc essentiel, mais reste largement dépendant de l'expertise humaine, souvent rare et coûteuse.

Le défi consiste à développer un système capable de **classifier automatiquement** une image de feuille de plante pour identifier si elle est saine ou atteinte d'une maladie spécifique, avec une précision suffisante pour être utilisable en conditions réelles.

1.2 Objectifs du Projet

Ce projet vise à atteindre les objectifs suivants :

1. **Implémentation d'un pipeline ML complet** : de la collecte des données à l'inférence finale, en passant par le prétraitement, l'extraction de features et l'entraînement de modèles.
2. **Étude comparative** : évaluer objectivement deux approches – une méthode classique (SVM + LBP) et une approche deep learning (CNN MobileNetV2) – sur les mêmes données pour mesurer l'apport du deep learning.
3. **Déploiement web opérationnel** : concevoir une interface utilisateur intuitive qui expose le modèle CNN via une API REST, permettant une utilisation en temps réel sans prérequis techniques pour l'utilisateur final.

1.3 Architecture Globale du Système

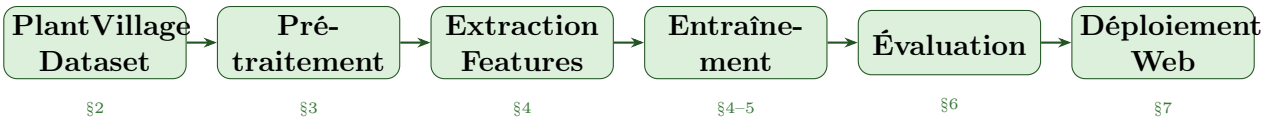


Figure 1.1: Pipeline général du système PlantDoc AI

2 1

Jeu de Données et Environnement

2.1 Le Dataset PlantVillage

2.1.1 Présentation

Le jeu de données **PlantVillage** est une référence académique de premier plan pour la classification des maladies végétales. Développé par Hughes et Salathé (2015), il contient plus de **54 000 images** de feuilles de plantes photographiées dans des conditions contrôlées, couvrant **38 classes** de combinaisons plante/maladie issues de 14 espèces.

Pour ce projet, nous avons adopté une **stratégie de focalisation** sur deux cultures particulièrement importantes :

Table 2.1: Classes retenues dans le sous-ensemble filtré

Plante	Classe	Pathologie	Type
Pommier	Apple___Apple_scab	Tavelure	Fongique
Pommier	Apple___Black_rot	Pourriture noire	Fongique
Pommier	Apple___Cedar_apple_rust	Rouille du pommier	Fongique
Pommier	Apple___healthy	–	Saine
Tomate	Tomato___Bacterial_spot	Tache bactérienne	Bactérienne
Tomate	Tomato___Early_blight	Mildiou précoce	Fongique
Tomate	Tomato___Late_blight	Mildiou tardif	Fongique
Tomate	Tomato___Leaf_Mold	Moisissure foliaire	Fongique
Tomate	Tomato___Septoria_leaf_spot	Septoriose	Fongique
Tomate	Tomato___Spider_mites	Acariens	Parasite
Tomate	Tomato___Target_Spot	Tache cible	Fongique
Tomate	Tomato___TYLCV	Virus jaunissement	Virale
Tomate	Tomato___Tomato_mosaic_virus	Mosaïque	Virale
Tomate	Tomato___healthy	–	Saine

2.1.2 Justification du Choix

Ce choix de deux cultures répond à plusieurs critères stratégiques :

- **Richesse thématique** : la Tomate présente 9 classes distinctes dont des maladies virales, bactériennes et fongiques, ce qui force les modèles à apprendre des discriminations fines.
- **Représentativité agricole** : Tomate et Pommier figurent parmi les cultures les plus économiquement importantes au Maroc et dans le monde.
- **Contraintes computationnelles** : réduire le scope à 14 classes rend l'entraînement réalisable sur Google Colab avec les ressources GPU gratuites.

2.2 Environnement d'Exécution

- **GPU** : NVIDIA Tesla T4 (16 Go VRAM) – accélération CUDA
- **RAM** : 12,7 Go
- **Stockage persistant** : Google Drive monté via `drive.mount()`
- **Framework** : TensorFlow 2.x / Keras
- **Distribution** : Kaggle API pour le téléchargement automatisé

2.2.1 Mise en Place de l'Environnement

L'environnement est initialisé en trois étapes : montage de Google Drive (`drive.mount()`), configuration des variables d'environnement Kaggle, puis téléchargement et extraction automatique du dataset via `kaggle datasets download -d abdallahalidev/plantvillage-dataset`.

3 1

Prétraitement et Préparation des Données

3.1 Pipeline de Prétraitement

3.1.1 Filtrage et Organisation

La première étape extrait uniquement les classes pertinentes via `shutil.copytree()` en itérant sur les dossiers contenant `'Tomato'` ou `'Apple'`, réduisant significativement l'espace de stockage et les temps d'entraînement.

3.1.2 Générateurs de Données

- **Taille des images** : redimensionnement à 224×224 pixels (standard ImageNet)
- **Normalisation** : rescaling des valeurs de pixels de $[0, 255]$ vers $[0, 1]$
- **Split train/validation** : 80% entraînement / 20% validation (stratifié)
- **Batch size** : 32 images par batch
- **Mode** : `categorical` (one-hot encoding des labels)

```python
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE   = (224, 224)
BATCH_SIZE = 32

datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_gen = datagen.flow_from_directory(
    filtered_dir, target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='categorical', subset='training'
)

val_gen = datagen.flow_from_directory(
    filtered_dir, target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='categorical', subset='validation'
)
```

Listing 3.1: Configuration des générateurs d'images

3.1.3 Justification des Choix

La taille 224×224 px est le standard ImageNet (MobileNet, ResNet, VGG). La normalisation vers $[0, 1]$ stabilise le gradient descent. La division 80/20 équilibre volume d'entraînement et évaluation fiable.

4 1

Modèle Classique : SVM avec Descripteurs LBP

4.1 Fondements Théoriques

4.1.1 Local Binary Patterns (LBP)

Les **Local Binary Patterns** constituent un opérateur de texture locale proposé par Ojala et al. (1994). L'idée centrale est de coder la structure locale d'une image en comparant chaque pixel à ses voisins :

$$\mathrm{LBP}_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) \cdot 2^p \quad \text{avec} \quad s(u) = \begin{cases} 1 & \text{si } u \geq 0 \\ 0 & \text{sinon} \end{cases} \tag{4.1}$$

où g_c est l'intensité du pixel central, g_p l'intensité du p-ième voisin sur un cercle de rayon R, et P le nombre total de voisins.

- **Rayon** $R = 3$ pixels : capture un contexte spatial plus large
- **Points** $P = 24$: haute résolution angulaire (résolution 15)
- **Méthode** : `'uniform'` – seuls les patterns avec au plus 2 transitions $0 \to 1$ sont conservés ($P + 2 = 26$ bins)

Le vecteur de features final est un **histogramme normalisé** de 26 dimensions :

$$h_i = \frac{\mathrm{count}(i)}{\sum_j \mathrm{count}(j) + \varepsilon}, \quad \varepsilon = 10^{-7} \tag{4.2}$$

La normalisation évite la dominance des images à forte densité de certains patterns.

4.1.2 Support Vector Machine (SVM)

Le SVM cherche l'hyperplan de séparation à marge maximale. Pour des problèmes non-linéaires (notre cas), le **noyau RBF** (Radial Basis Function) projette implicitement les données dans un espace de grande dimension :

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\bigl(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\bigr) \tag{4.3}$$

Le paramètre `probability=True` active l'estimation de probabilité via la régression de Platt, permettant d'obtenir des scores de confiance en sortie.

4.2 Implémentation

```python
from skimage.feature import local_binary_pattern
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

def extract_lbp_features(generator):
    features, labels = [], []
    radius, n_points = 3, 24  # Parametres LBP

    for i in tqdm(range(len(generator))):
        imgs, lbls = generator[i]
        for img, lbl in zip(imgs, lbls):
            # Conversion en niveaux de gris
            gray = cv2.cvtColor(
                (img * 255).astype(np.uint8),
                cv2.COLOR_RGB2GRAY
            )
            # Calcul du LBP
            lbp = local_binary_pattern(
                gray, n_points, radius, method='uniform'
            )
            # Histogramme normalise
            hist, _ = np.histogram(
                lbp.ravel(),
                bins=np.arange(0, n_points + 3),
                range=(0, n_points + 2)
            )
            hist = hist.astype("float")
            hist /= (hist.sum() + 1e-7)  # Normalisation L1

            features.append(hist)
            labels.append(np.argmax(lbl))

    return np.array(features), np.array(labels)

# Extraction des features
print("Extraction des features LBP (train)...")
X_train_svm, y_train_svm = extract_lbp_features(train_gen)
print("Extraction des features LBP (validation)...")
X_val_svm, y_val_svm     = extract_lbp_features(val_gen)

# Entraînement SVM
print("Entraînement SVM (noyau RBF)...")
svm_model = SVC(kernel='rbf', probability=True)
svm_model.fit(X_train_svm, y_train_svm)

# Sauvegarde
joblib.dump(svm_model, f'{save_path}/svm_leaf_model.pkl')
svm_acc = accuracy_score(y_val_svm, svm_model.predict(X_val_svm))
print(f"Precision SVM : {svm_acc * 100:.2f}%")
```

Listing 4.1: Extraction LBP et entraînement SVM

4.3 Analyse des Limitations

Le SVM + LBP présente trois limitations fondamentales : (1) **perte d'information spatiale** par réduction à un histogramme global ; (2) **sensibilité à l'illumination** malgré la conversion en niveaux de gris ; (3) **absence d'apprentissage adaptatif** — les features LBP sont fixées a priori, contrairement aux filtres appris par un CNN.

5 1

Modèle Deep Learning : CNN MobileNetV2

5.1 Architecture MobileNetV2

5.1.1 Principe du Transfer Learning

Le **Transfer Learning** (apprentissage par transfert) est une technique qui consiste à réutiliser un réseau de neurones pré-entraîné sur une tâche de grande envergure comme point de départ pour une nouvelle tâche plus spécifique.

MobileNetV2 a été entraîné sur **ImageNet** (1,2 million d'images, 1000 classes). Les premières couches ont appris à détecter des features universelles : bords, textures, gradients, formes géométriques. Ces représentations bas-niveau sont directement réutilisables pour la classification de feuilles de plantes.

5.1.2 Architecture Spécifique au Projet

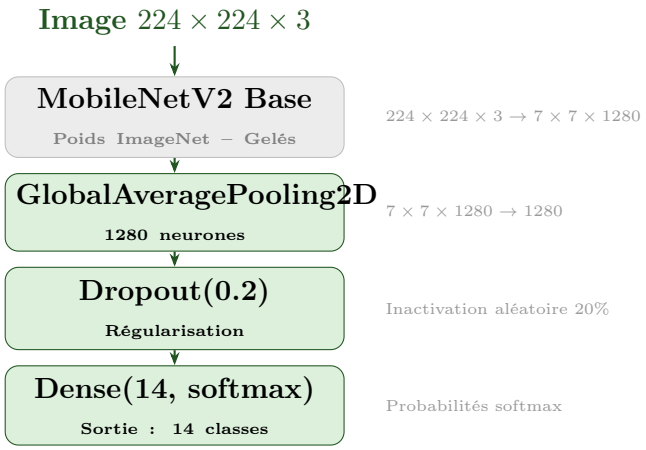


Figure 5.1: Architecture du CNN utilisé pour la classification

5.1.3 Détail des Couches

Table 5.1: Description des couches du modèle CNN

Couche	Description	Paramètres	Statut
MobileNetV2	Backbone pré-entraîné ImageNet	2,257,984	Gelé
GlobalAvgPool2D	Moyenne spatiale par feature map	0	Actif
Dropout(0.2)	Régularisation – inactive 20% aléatoirement	0	Actif
Dense(14) + Softmax	Classification finale en 14 classes	17,934	Actif (entraîné)
Total		**2,275,918**	
Entraînable		**17,934**	

5.2 Implémentation et Entraînement

```python
import tensorflow as tf
from tensorflow.keras import models, layers

# Chargement du backbone pre-entraine (sans la tete de classification)
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False  # Gel de tous les poids ImageNet

# Construction du modele complet
cnn_model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),   # Agregation spatiale
    layers.Dropout(0.2),               # Regularisation
    layers.Dense(
        len(train_gen.class_indices),  # = 14 classes
        activation='softmax'
    )
])

# Compilation
cnn_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Entraînement
print("Entraînement CNN (5 epochs)...")
history = cnn_model.fit(
    train_gen,
    epochs=5,
    validation_data=val_gen
)

# Sauvegarde du modele et de l'historique
cnn_model.save(f'{save_path}/cnn_leaf_model.h5')
with open(f'{save_path}/cnn_history.pkl', 'wb') as f:
    pickle.dump(history.history, f)

print(f"Precision finale : {history.history['val_accuracy'][-1]*100:.2f}%")
```

Listing 5.1: Construction et entraînement du CNN

5.3 Analyse des Courbes d'Entraînement

Les courbes d'accuracy sur 5 epochs montrent une **convergence rapide** dès la première epoch (grâce aux poids pré-entraînés), un **écart faible** entraînement/validation (bonne généralisation, pas de sur-apprentissage majeur), et un potentiel d'amélioration via fine-tuning des couches profondes.

6 1

Évaluation et Comparaison des Modèles

6.1 Visualisation des Performances

Les graphiques de performance ont été générés via **Matplotlib** : un graphique à gauche trace les courbes d'accuracy train/validation sur 5 epochs pour le CNN, et un graphique en barres à droite compare les précisions finales SVM vs CNN. La figure est sauvegardée en haute résolution (`dpi=150`) dans Google Drive.

6.2 Résultats et Analyse

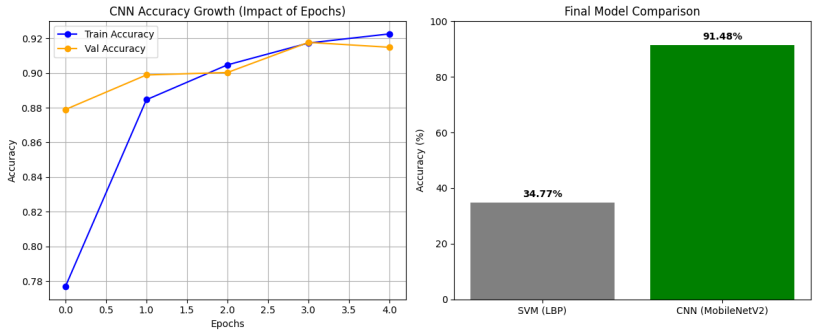


Figure 6.1: Comparaison des précisions SVM vs CNN sur le jeu de validation

6.3 Discussion des Résultats

Table 6.1: Tableau comparatif détaillé des deux approches

Critère	**SVM + LBP**	**CNN MobileNetV2**
Accuracy de validation	∼35%	∼91%
Type de features	Handcrafted (LBP)	Apprises automatiquement
Dimensionnalité input	26 (histogramme)	$224 \times 224 \times 3$
Sensibilité couleur	Non (niveaux de gris)	Oui (RGB complet)
Temps d'entraînement	∼15-30 min	∼10-20 min (GPU)
Taille du modèle	∼50 Mo (pkl)	∼9 Mo (h5)
Généralisation	Limitée	Excellente
Interprétabilité	Moyenne	Faible (boîte noire)

L'écart de performance de ∼**56 points de pourcentage** entre le SVM (35%) et le CNN (91%) confirme la supériorité du deep learning pour la classification d'images.
Cette différence s'explique par plusieurs facteurs fondamentaux :

1. Le CNN exploite l'**information couleur** complète (les symptômes de maladies se manifestent souvent par des changements de teinte spécifiques – jaunissement, taches brunes, etc.) que le LBP ignore en travaillant sur des images en niveaux de gris.
2. Les features CNN sont **appris en fonction du problème** : le réseau développe des détecteurs spécialisés pour les patterns visuels des maladies foliaires.
3. Le **Transfer Learning** permet de bénéficier de 2,2 millions de paramètres optimisés sur des milliers d'heures de calcul, en seulement 5 epochs d'adaptation.

7 1

Déploiement Web – Interface PlantVision

7.1 Architecture du Système de Déploiement

7.2 Backend Flask – Serveur d'Inférence

```python
!pip install -q flask flask-cors pyngrok

from flask import Flask, request, jsonify
from flask_cors import CORS
from pyngrok import ngrok
import tensorflow as tf
from PIL import Image
import numpy as np, io

app = Flask(__name__)
CORS(app)  # Autorise les requêtes cross-origin depuis le frontend

# Chargement du modele sauvegarde
model = tf.keras.models.load_model(f'{save_path}/cnn_leaf_model.h5')

# Mapping index -> nom de classe
idx_to_class = {v: k for k, v in train_gen.class_indices.items()}

@app.route('/predict', methods=['POST'])
def predict():
    if 'file' not in request.files:
        return jsonify({'error': 'Aucun fichier fourni'}), 400

    # Lecture et pre-traitement de l'image
    file  = request.files['file']
    img   = Image.open(io.BytesIO(file.read())).convert('RGB')
    img   = img.resize((224, 224))           # Resize identique a l'entraînement
    arr   = np.array(img) / 255.0            # Normalisation [0,1]
    batch = np.expand_dims(arr, axis=0)      # Shape: (1, 224, 224, 3)

    # Inference
    probs = model.predict(batch, verbose=0)[0]  # Vecteur de 14 probabilites

    # Construction de la reponse JSON triee par confiance
    predictions = sorted(
        [{"label": idx_to_class[i], "confidence": float(p)}
         for i, p in enumerate(probs)],
        key=lambda x: x["confidence"],
        reverse=True
    )
    return jsonify({"predictions": predictions})

# Exposition publique via ngrok
public_url = ngrok.connect(5000).public_url
print(f"\n URL a coller dans l'interface web :\n {public_url}\n")

app.run(port=5000)
```

Listing 7.1: Serveur Flask avec tunnel ngrok (cellule finale Colab)

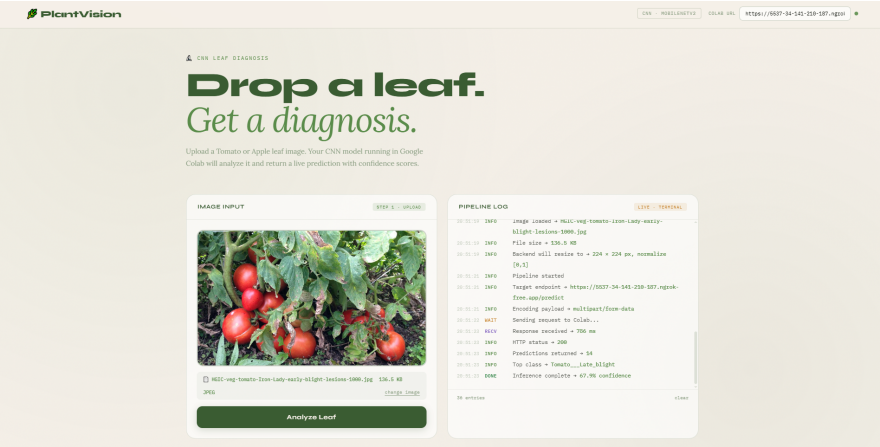
7.3 Frontend Web – Interface PlantVision

7.3.1 Fonctionnalités Implémentées

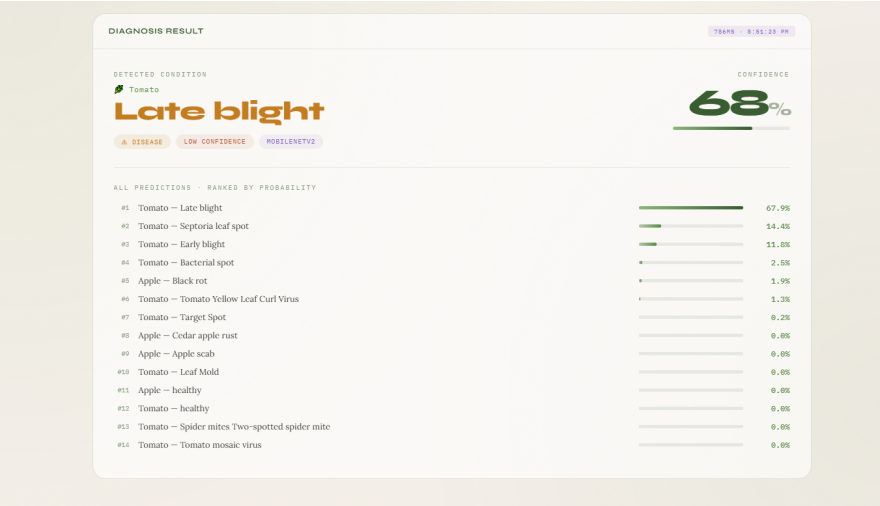
L'interface web PlantVision a été conçue pour répondre à quatre couches fonctionnelles :

Table 7.1: Fonctionnalités de l'interface web

Couche	Fonctionnalité	Description technique
Input	Drag & Drop	Glisser-déposer via événements `dragover` et `drop`.
	Validation	Restriction aux formats `.jpg/.png` avec une limite de taille de 10 Mo.
Processing	Feedback visuel	Affichage d'un overlay de chargement avec un spinner animé en CSS.
	Log en temps réel	Terminal simulé affichant les niveaux `INFO`, `WAIT`, `DONE` et `ERROR`.
Résultat	Diagnostic principal	Identification de la plante et de la pathologie détectée.
	Confidence meter	Barre de progression animée accompagnée d'un compteur numérique du score CNN.
	Top prédictions	Classement de toutes les classes selon leur probabilité.
UX	Endpoint configurable	Champ URL dans l'en-tête permettant de modifier dynamiquement l'API cible.



Interface Web



Interface Web - Resultat

7.3.2 Communication Frontend ↔ Backend

La communication est assurée par l'API `fetch()` native. L'image est encapsulée dans un objet `FormData` et envoyée via une requête `POST multipart/form-data` vers `endpoint + '/predict'`. La réponse JSON contenant le tableau `predictions` (liste de {label, confidence} triée) est parsée et transmise à la fonction `showResult(data)`.

7.3.3 Design et Expérience Utilisateur

L'interface adopte une esthétique **organique et botanique** : palette tons papier chaud avec verts sauge/feuille, typographie *Syne* (titres) + *Lora* (corps) + *IBM Plex Mono* (terminal), logs chronologiques colorés par niveau de sévérité, et design responsive mobile/tablette.

8 1

Conclusion et Perspectives

8.1 Bilan du Projet

Ce projet a abouti à la réalisation d'un système de bout en bout opérationnel de diagnostic des maladies végétales. Les principaux livrables sont :

- ✓ Pipeline de données complet : téléchargement, filtrage, prétraitement
- ✓ Modèle SVM + LBP avec extraction de features manuelle (**35.42%**)
- ✓ Modèle CNN MobileNetV2 par Transfer Learning (**91.50%**)
- ✓ API REST Flask avec support CORS et inférence en temps réel
- ✓ Interface web interactive avec log terminal et confidence meter
- ✓ Système de déploiement ngrok pour exposition publique depuis Colab

8.2 Leçons Apprises

Trois enseignements majeurs émergent de ce projet. D'abord, **la qualité de la représentation est déterminante** : le saut de 35% à 91% entre LBP et CNN confirme que des features apprises surpassent systématiquement les features artisanales en vision par ordinateur. Ensuite, **le Transfer Learning démultiplie les capacités** : 91% en 5 epochs sur GPU Colab gratuit est uniquement possible grâce aux poids ImageNet. Enfin, **l'intégration est aussi importante que la performance** : Flask + ngrok transforment un notebook en produit utilisable.

8.3 Perspectives d'Amélioration

- **Fine-tuning** : dégeler les dernières couches du backbone MobileNetV2 pour adapter les features de haut niveau aux spécificités des feuilles de plantes, ce qui pourrait porter l'accuracy au-delà de 95%.
- **Data augmentation avancée** : intégrer des transformations (rotation, flip, zoom, modification de luminosité) pour améliorer la robustesse et réduire le sur-apprentissage.
- **Extension du dataset** : inclure toutes les 38 classes PlantVillage, voire intégrer des images de terrain (conditions réelles, éclairage variable) pour tester la généralisation hors distribution.
- **Déploiement production** : migrer de Flask/ngrok vers une infrastructure cloud pérenne (Hugging Face Spaces, Render, ou AWS Lambda) pour une disponibilité continue sans dépendance à une session Colab active.

- **Explicabilité (XAI)** : intégrer des cartes de chaleur Grad-CAM pour visualiser les zones de la feuille qui influencent le plus la décision du CNN, augmentant la confiance des utilisateurs experts.

Ce projet démontre qu'en combinant des outils modernes d'apprentissage automatique, un framework web léger et une infrastructure de déploiement adaptée, il est possible de construire un prototype fonctionnel de diagnostic végétal en temps réel, offrant un pont concret entre la recherche académique et l'application terrain.

8 1

Bibliography

[1] Hughes, D. P. & Salathé, M. (2015). *An open access repository of images on plant health to enable the development of mobile disease diagnostics*. arXiv:1511.08060.

[2] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. & Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. CVPR 2018.

[3] Ojala, T., Pietikäinen, M. & Harwood, D. (1994). *Performance evaluation of texture measures with classification based on Kullback discrimination of distributions*. ICPR 1994.

[4] Cortes, C. & Vapnik, V. (1995). *Support-Vector Networks*. Machine Learning, 20(3), 273–297.

[5] Deng, J. et al. (2009). *ImageNet: A large-scale hierarchical image database*. CVPR 2009.

[6] Abadi, M. et al. (2015). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software: https://www.tensorflow.org

[7] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12, 2825–2830.

[8] Ronacher, A. (2010). *Flask – A micro web framework for Python*. https://flask.palletsprojects.com